The system implements an end-to-end, GitOps-driven microservices architecture on Kubernetes designed for automated deployment, seamless internal service communication, and real-time observability.

Architecture Overview & Components

- **Microservices Application (ocr-app namespace):**
  - **API Gateway (api-gateway):** Acts as the entry point handling external client requests. It forwards optical character recognition (OCR) inference requests to the internal model server using Kubernetes DNS.
  - **OCR Model Server (ocr-model-server):** Executes model inference using Tesseract OCR.
- **GitOps Deployment Engine (argocd namespace):**
  - **AppProject (ocr-project):** Establishes logical security boundaries, target namespace permissions (ocr-app, monitoring), and repository whitelisting.
  - **ApplicationSet (ocr-services-appset):** Dynamically generates and syncs ArgoCD Application resources for all services defined under charts/.
  - **Application (ocr-monitoring-stack):** Deploys the standalone kube-prometheus-stack chart directly from upstream repositories.
- **Observability & Metrics (monitoring namespace):**
  - **kube-prometheus-stack:** Deploys Prometheus and Grafana for monitoring.
  - **PodMonitor (ocr-podmonitor.yaml):** Scrapes custom /metrics endpoints exposed on port 8080 of the model server every 5 seconds

```
## 📋 Prerequisites

Before setting up the project, ensure you have the following installed
on your machine:

* [Docker](https://docs.docker.com/get-docker/)
* [Minikube](https://minikube.sigs.k8s.io/docs/start/)
* [kubectl](https://kubernetes.io/docs/tasks/tools/)
* [Helm v3](https://helm.sh/docs/intro/install/)
* [pipx](https://github.com/pypa/pipx)

---

## 🚀 Getting Started

###  Clone the Repository
```

```bash
git clone
[https://github.com/Azmeth23/cluster-system-ocr.git](https://github.com/Azmeth23/cluster-system-ocr.git)
cd cluster-system-ocr

# Deployment Guide

This guide walks you through setting up the infrastructure, deploying the application, and verifying that all services are running successfully.

---

## 1. Set Up Infrastructure

Run the infrastructure setup script to configure the local Kubernetes environment and install all required dependencies.

```bash
cd infrastructure
./infrastructure-setup.sh
```

---

2. Deploy GitOps Resources (ArgoCD)

Apply the ArgoCD resources to enable GitOps-based deployment and synchronization.

```bash
kubectl apply -f argocd-resources.yaml
```

Helm Charts & Image Management

Package Helm Charts

Package both application Helm charts.
```

```bash
helm package api-gateway
helm package ocr-model
```

Login to Docker Hub OCI Registry

Authenticate with Docker Hub before pushing the Helm charts.

```bash
helm registry login registry-1.docker.io -u azmeth07
```

Push Helm Charts

Push the packaged charts to the OCI registry.

```bash
helm push api-gateway-0.1.0.tgz oci://registry-1.docker.io/azmeth07

helm push ocr-model-0.1.0.tgz oci://registry-1.docker.io/azmeth07
```

Deploy to Minikube

1. Create Docker Image Pull Secret

If your Docker images are stored in a private repository, create a Kubernetes image pull secret.

```bash
kubectl create secret docker-registry dockerhub-secret \
 --docker-username=YOUR_DOCKER_USERNAME \
 --docker-password=YOUR_DOCKER_PASSWORD \
 --docker-email=YOUR_EMAIL
```

---

2. Create the Application Namespace

```bash
kubectl create namespace ocr-app
```

---

3. Deploy the OCR Model Server

```bash
helm install ocr-model-server ./charts/ocr-model-server -n ocr-app
```

---

4. Deploy the API Gateway

```bash
helm install ocr-api-gateway ./charts/api-gateway -n ocr-app
```

Verification

Verify that all application components have been deployed successfully.

Check Running Pods

```bash
kubectl get pods -n ocr-app
```

Expected output:

- OCR Model Server pod
- API Gateway pod
- All pods should have a **Running** status.

Check Services

```bash
```

```
kubectl get svc -n ocr-app
```

This command lists all Kubernetes services exposed within the `ocr-app` namespace.

View Application Logs

To inspect logs for a specific pod:

```bash
kubectl logs -f <pod-name> -n ocr-app
```

Replace `<pod-name>` with the name of the pod you want to monitor.

Deployment Summary

The deployment process consists of:

1. Setting up the infrastructure.
2. Deploying ArgoCD GitOps resources.
3. Packaging Helm charts.
4. Publishing charts to the OCI registry.
5. Creating the Docker image pull secret.
6. Creating the application namespace.
7. Deploying the OCR Model Server.
8. Deploying the FastAPI API Gateway.
9. Verifying pods, services, and application logs.

Containerization strategy

Base image selection: python:3.11-slim version is used because it’s capabilities were fit for a production level app as this. While default version can also be used used here, it possess utilities not necessary for the build.

Security considertions: By presetting values for certain tasks that may arrive during poetries installation it arranges the build to take place smoothly without stalling/crashing

Build optimization: The dependency files for the poetry environment were copied before the codes since there changes are not frequent and so make building the docker image fast

Possible improvement, using a multistage build by instaling the heavy build tools in stage 1, building the app, and then copy only the final compiled artifacts.